

Email Attachment Extractor

Automated PDF / Image Extraction and Reply Pipeline

Live demo : <https://hdfc-email-extractor.onrender.com>

Source : <https://github.com/zAcKeR-KrAcKeR/hdfc-email-extractor>

1. Overview

This system monitors a Gmail inbox via IMAP, detects incoming emails that carry PDF or image attachments, extracts structured data from those attachments using OCR and regex pattern matching, and automatically replies to the original sender in the same email thread with the extracted fields. No LLM or third-party AI API is required.

The pipeline is designed around five discrete, independently testable stages so that any individual component (the email reader, the OCR extractor, the field parser, or the SMTP sender) can be swapped or tested in isolation.

2. Pipeline Architecture

Each incoming email passes through the following five stages in sequence:

1	Fetch Email	Connect via IMAP SSL. Search last 3 days. Process up to 5 per run.
2	Read Attachment	Walk the MIME tree. Accept .pdf .png .jpg .jpeg .tiff .bmp.
3	OCR Extraction	Digital PDFs: pdfplumber. Scanned PDFs / images: Tesseract OCR.
4	Field Parsing	Regex patterns: name, email, phone, date, amount, PAN, account, skills.
5	Auto Reply	In-Reply-To + References headers keep it threaded. SMTP port 587.

3. Idempotency

Each processed Message-ID is recorded in a local SQLite database (processed_messages.sqlite3). Before processing any email the pipeline checks this ledger. If the Message-ID is already present the email is skipped. This guarantees that a re-run or crash-recovery never sends duplicate replies, even if the service restarts mid-batch.

4. Extracted Fields

The regex parser attempts to extract the following fields from every document:

name	Labeled name field (Name:, Applicant:, Candidate:)
email	Email address matching standard RFC 5322 pattern
phone	10-digit Indian or international phone number
date	Common formats: DD/MM/YYYY, YYYY-MM-DD, Month DD YYYY
amount	Currency values preceded by Rs., INR, USD, \$ or keywords

pan_number	Indian PAN: 5 uppercase letters + 4 digits + 1 uppercase letter
account_number	Labeled account number field
reference_number	Labeled reference / ref / ref no field
skills	Text following a Skills: or Technologies: heading
raw_text_preview	First 300 characters of extracted text for audit purposes

5. Setup and Configuration

All configuration is supplied via environment variables. Nothing is hardcoded.

Variable	Description	Example
IMAP_HOST	IMAP server hostname	imap.gmail.com
SMTP_HOST	SMTP server hostname	smtp.gmail.com
EMAIL_USER	Gmail address to monitor	your@gmail.com
EMAIL_PASS	Gmail App Password (16 chars, Google Account > Security > App Passwords) xxxx	xxxx

6. Gmail Prerequisites

1. Enable 2-Step Verification on the Gmail account.
2. Go to myaccount.google.com/apppasswords and generate an App Password. Use the 16-character key as EMAIL_PASS (not your real Gmail password).
3. In Gmail Settings > See all settings > Forwarding and POP/IMAP, set IMAP access to Enabled.

7. Running Locally

Install Python 3.10+, then:

```
pip install -r requirements.txt
cp .env.example .env          # fill in your values
python app.py                 # starts Flask on http://localhost:5000
```

Open <http://localhost:5000>. The dashboard shows the pipeline diagram, the monitored inbox address, and the latest processed email with extracted fields. Click Run Now to trigger an immediate inbox scan without waiting for the 2-minute auto-scan.

8. Deploying to Render

The repository includes a render.yaml blueprint. Steps:

1. Push the repo to GitHub.
2. Go to dashboard.render.com > New > Blueprint > select the repo.
3. Add environment variables (IMAP_HOST, SMTP_HOST, EMAIL_USER, EMAIL_PASS) in the service Environment tab.
4. Render builds and deploys automatically.

Render does not block outbound SMTP (port 587 STARTTLS), so email replies are delivered without any additional network configuration.

9. Web Dashboard

The Flask web server exposes three endpoints:

GET / Renders the dashboard: pipeline diagram, monitored inbox, and the most recently processed email with extracted fields.

POST /run Triggers an immediate inbox scan. Returns the dashboard with a result summary.

GET /status Returns {"status": "ok"} used as a health-check by Render.

10. Security Notes

- All credentials are loaded from environment variables. Nothing is hardcoded.
- The .env file and SQLite database are excluded from version control via .gitignore.
- Gmail App Passwords are scoped to a single application and can be revoked independently.
- Extracted data is stored only in the SQLite run_log table and is not transmitted to any third party.
- Replies use In-Reply-To and References headers, sent only to the original sender.

11. Technology Stack

Flask	Web framework and dashboard UI
APScheduler	Background scheduler (inbox scan every 2 minutes)
imaplib	Python standard library - IMAP email fetching
smtplib	Python standard library - SMTP email sending
pdfplumber	PDF text layer extraction for digital PDFs
pytesseract	OCR for scanned PDFs and image attachments (optional)
SQLite	Idempotency ledger and run log
Render	Cloud hosting - always-on, no SMTP restrictions
gunicorn	Production WSGI server